

# Laboratório - Aula 12

October 5, 2026

## 0.1 EXERCÍCIO 1: CÁLCULO MANUAL DE ENTROPIA E GANHO DE INFORMAÇÃO

```
[8]: import numpy as np
def calcular_entropia(y):
    if len(y) == 0:
        return 0.0
    _, counts = np.unique(y, return_counts=True)
    probs = counts / len(y)
    probs = probs[probs > 0]
    return float(-np.sum(probs * np.log2(probs)))

def calcular_entropia_e_ganho(X_column, y):
    X_column = np.array(X_column)
    y = np.array(y)
    H_y = calcular_entropia(y)
    values, counts = np.unique(X_column, return_counts=True)
    weighted_H = 0.0
    total = len(y)
    for val, count in zip(values, counts):
        sub_y = y[X_column == val]
        weighted_H += (count / total) * calcular_entropia(sub_y)
    IG = H_y - weighted_H
    return round(H_y, 4), round(float(IG), 4)

y_ex = [0, 0, 0, 1, 1, 1, 1, 1]
X_ex = ['Ruim', 'Ruim', 'Bom', 'Bom', 'Bom', 'Bom', 'Ruim', 'Bom']
h_val, ig_val = calcular_entropia_e_ganho(X_ex, y_ex)
print(f"Entropia Inicial H(y): {h_val} bits")
print(f"Ganho de Informação IG(y, X): {ig_val} bits")
```

Entropia Inicial H(y): 0.9544 bits

Ganho de Informação IG(y, X): 0.1589 bits

## 0.2 EXERCÍCIO 2: INDUÇÃO, REGULARIZAÇÃO E VISUALIZAÇÃO COM PLOT\_TREE (GINI)

```
[17]: from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
from sklearn.metrics import accuracy_score
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
def treinar_e_visualizar_arvore(X_train, X_test, y_train, y_test,
    ↪feature_names, class_names, criterion='gini', max_depth=3,
    ↪min_samples_leaf=5):
    clf = DecisionTreeClassifier(
        criterion=criterion,
        max_depth=max_depth,
        min_samples_leaf=min_samples_leaf,
        random_state=42
    )
    clf.fit(X_train, y_train)

    regras = export_text(clf, feature_names=list(feature_names))
    print("--- Regras Lógicas da Árvore ---")
    print(regras)

    fig, ax = plt.subplots(figsize=(10, 6), dpi=150)
    plot_tree(
        clf,
        feature_names=feature_names,
        filled=True,
        rounded=True,
        fontsize=9,
        ax=ax
    )
    plt.title("Árvore de Decisão - Dataset Iris (Gini)", fontsize=12,
    ↪fontweight='bold')
    plt.tight_layout()
    plt.savefig("arvore_decisao_iris.png")
    plt.show()
    plt.close()

    y_pred = clf.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    return {
        "accuracy": round(float(acc), 4),
        "node_count": int(clf.tree_.node_count),
        "max_depth": int(clf.get_depth()),
        "n_leaves": int(clf.get_n_leaves())
    }
```

```

iris = load_iris()
X_tr, X_te, y_tr, y_te = train_test_split(
    iris.data, iris.target, test_size=0.3, random_state=42, stratify=iris.target
)
res = treinar_e_visualizar_arvore(
    X_tr, X_te, y_tr, y_te,
    feature_names=iris.feature_names,
    class_names=iris.target_names
)
print("Resultado da Árvore Regularizada:", res)

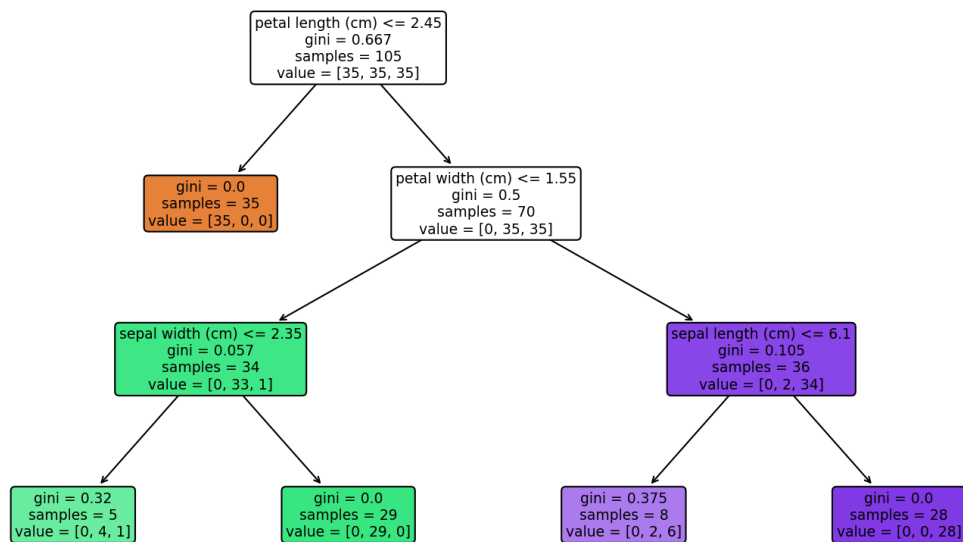
```

```

--- Regras Lógicas da Árvore ---
|--- petal length (cm) <= 2.45
|   |--- class: 0
|--- petal length (cm) > 2.45
|   |--- petal width (cm) <= 1.55
|       |--- sepal width (cm) <= 2.35
|       |   |--- class: 1
|       |   |--- sepal width (cm) > 2.35
|       |   |--- class: 1
|       |--- petal width (cm) > 1.55
|           |--- sepal length (cm) <= 6.10
|           |--- class: 2
|           |--- sepal length (cm) > 6.10
|           |--- class: 2

```

**Árvore de Decisão — Dataset Iris (Gini)**



Resultado da Árvore Regularizada: {'accuracy': 0.8889, 'node\_count': 9, 'max\_depth': 3, 'n\_leaves': 5}

### 0.3 EXERCÍCIO 3: ÁRVORES DE REGRESSÃO COM DECISIONTREERE- GRESSOR

```
[18]: from sklearn.tree import DecisionTreeRegressor, plot_tree
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt

def treinar_arvore_regressao(X_train, X_test, y_train, y_test, max_depth=4):
    reg = DecisionTreeRegressor(max_depth=max_depth, random_state=42)
    reg.fit(X_train, y_train)
    y_pred = reg.predict(X_test)

    mse = mean_squared_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)
    n_leaves = reg.get_n_leaves()

    return reg, round(float(mse), 4), round(float(r2), 4), int(n_leaves)

X_reg, y_reg = make_regression(n_samples=150, n_features=5, noise=10.0,
    ↪random_state=42)
Xr_tr, Xr_te, yr_tr, yr_te = train_test_split(X_reg, y_reg, test_size=0.3,
    ↪random_state=42)
modelo_arvore, mse_val, r2_val, leaves = treinar_arvore_regressao(Xr_tr, Xr_te,
    ↪yr_tr, yr_te)
print(f"MSE Teste: {mse_val} | R2 Teste: {r2_val} | Folhas: {leaves}")

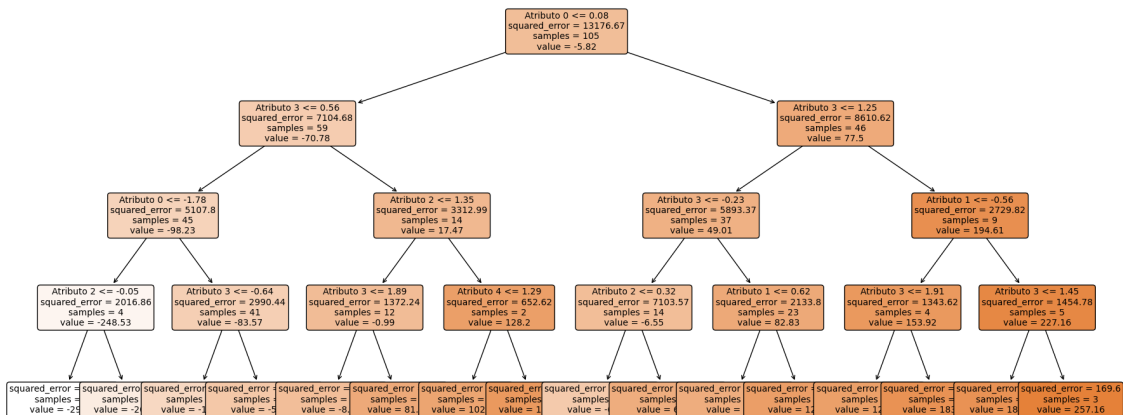
plt.figure(figsize=(18, 8))

plot_tree(modelo_arvore,
          filled=True,
          rounded=True,
          feature_names=[f"Atributo {i}" for i in range(X_reg.shape[1])],
          precision=2,
          fontsize=10)

plt.title("Estrutura Interna da Árvore de Regressão", fontsize=14, pad=20)
plt.tight_layout()
plt.show()
```

MSE Teste: 4198.0589 | R2 Teste: 0.5805 | Folhas: 16

Estrutura Interna da Árvore de Regressão



## 0.4 EXERCÍCIO 4: EXTRAÇÃO E RANKING DA IMPORTÂNCIA DE ATRIBUTOS

```
[21]: from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import load_iris
def rank_importancia_atributos(X, y, feature_names, max_depth=4):
    clf = DecisionTreeClassifier(max_depth=max_depth, random_state=42)
    clf.fit(X, y)
    importances = clf.feature_importances_
    ranked = sorted(zip(feature_names, importances), key=lambda x: x[1],
    ↪reverse=True)
    return [(name, round(float(imp), 4)) for name, imp in ranked]

iris = load_iris()
ranking = rank_importancia_atributos(iris.data, iris.target, iris.feature_names)
print("Ranking de Importância dos Atributos:")
for feat, imp in ranking:
    print(f" - {feat:20s}: {imp}")
```

Ranking de Importância dos Atributos:

- petal length (cm) : 0.5582
- petal width (cm) : 0.4283
- sepal width (cm) : 0.0135
- sepal length (cm) : 0.0

## 0.5 EXERCÍCIO 5: PÓS-PODA POR CUSTO-COMPLEXIDADE

```
[22]: import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score
```

```

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

def otimizar_pos_poda_ccp(X_train, X_test, y_train, y_test):
    base_clf = DecisionTreeClassifier(random_state=42)
    base_clf.fit(X_train, y_train)
    path = base_clf.cost_complexity_pruning_path(X_train, y_train)
    ccp_alphas = path.ccp_alphas
    ccp_alphas = ccp_alphas[ccp_alphas >= 0]
    best_acc = -1.0
    best_alpha = 0.0
    best_node_count = 0
    best_clf = None

    for alpha in ccp_alphas:
        clf = DecisionTreeClassifier(ccp_alpha=alpha, random_state=42)
        clf.fit(X_train, y_train)
        y_pred = clf.predict(X_test)
        acc = accuracy_score(y_test, y_pred)
        if acc > best_acc:
            best_acc = acc
            best_alpha = alpha
            best_node_count = clf.tree_.node_count
            best_clf = clf
    return base_clf, best_clf, round(float(best_alpha), 4),
    round(float(best_acc), 4), int(best_node_count)

cancer = load_breast_cancer()
Xc_tr, Xc_te, yc_tr, yc_te = train_test_split(
    cancer.data, cancer.target, test_size=0.3, random_state=42, stratify=cancer.
    target
)
arvore_completa, arvore_podada, alpha_opt, acc_opt, nodes_opt =
    otimizar_pos_poda_ccp(Xc_tr, Xc_te, yc_tr, yc_te)

print(f"Melhor CCP Alpha: {alpha_opt} | Acurácia Otimizada: {acc_opt} | Nós
    Restantes: {nodes_opt}")

fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(24, 10))
plot_tree(arvore_completa,
    filled=True,
    rounded=True,
    feature_names=cancer.feature_names,
    class_names=cancer.target_names,
    ax=axes[0])
axes[0].set_title(f"Antes da Poda (Árvore Completa)\nNós: {arvore_completa.
    tree_.node_count}", fontsize=16, pad=15)

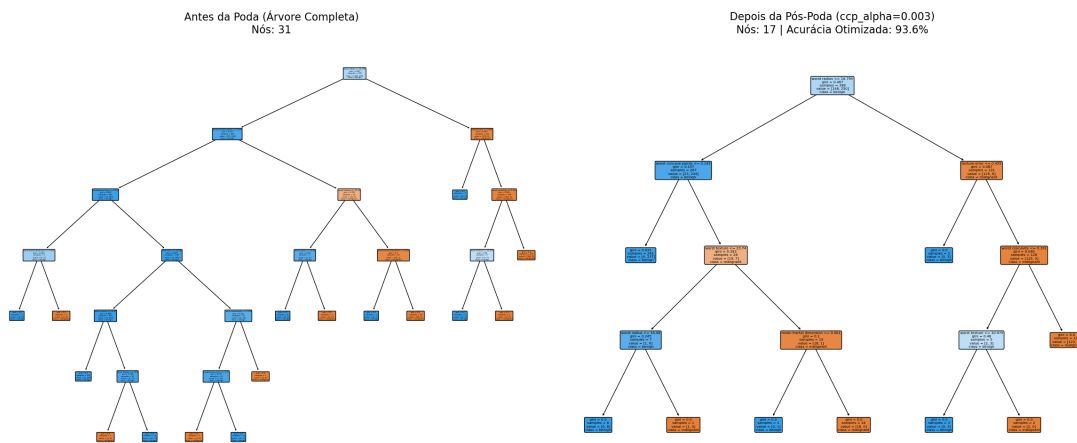
```

```

plot_tree(arvore_podada,
          filled=True,
          rounded=True,
          feature_names=cancer.feature_names,
          class_names=cancer.target_names,
          ax=axes[1])
axes[1].set_title(f"Depois da Pós-Poda (ccp_alpha={alpha_opt})\nNós:↳
↳{nodes_opt} | Acurácia Otimizada: {acc_opt*100:.1f}%", fontsize=16, pad=15)
plt.tight_layout()
plt.show()

```

Melhor CCP Alpha: 0.003 | Acurácia Otimizada: 0.9357 | Nós Restantes: 17



[ ]: